

ESTRATÉGIA DE TESTES E VERSIONAMENTO

Projeto Estacionamento - Parte 2

Responsável pela parte de testes: Pedro Henrique Monteiro Rodrigues

1. Objetivo da parte de testes

Esta documentação apresenta a estratégia de testes do protótipo do sistema de estacionamento. A entrega contempla 15 casos de teste estruturados e 15 testes unitários em JUnit, cobrindo cenários válidos, alternativos e de exceção. Também foram usados stubs para simular dependências e Mockito para demonstrar uso de mocks como desafio extra.

2. Relação com os padrões de projeto

Factory Method: testado pela criação de vagas VIP, elétricas, comuns e comportamento alternativo.

Adapter: testado no processamento de pagamentos via Pix, cartão e dinheiro.

Builder: testado na criação controlada de reservas.

State: testado pela atribuição e troca de estados da reserva.

Observer: testado pela notificação de observadores usando stub e mock.

3. Casos de teste estruturados

Nº	Caso de teste	Cenário	Entrada / ação	Resultado esperado
CT01	Criar vaga VIP	Válido	criarVaga("VIP")	Retornar uma instância de VagaVip.
CT02	Criar vaga elétrica	Válido	criarVaga("ELETRICA")	Retornar uma instância de VagaEletrica.
CT03	Criar vaga comum	Válido	criarVaga("COMUM")	Retornar uma instância de VagaComum.
CT04	Criar vaga com tipo desconhecido	Alternativo	criarVaga("GRANDE_PORTE")	Retornar VagaComum como comportamento padrão do protótipo.
CT05	Criar vaga com tipo nulo	Exceção	criarVaga(null)	Lançar IllegalArgumentException.
CT06	Processar pagamento Pix	Válido	PagamentoAdapter("PIX").processarPagamento(50.0)	Retornar true.

CT07	Processar pagamento Cartão	Válido	PagamentoAdapter("CA RTAO").processarPagamento(80.0)	Retornar true.
CT08	Processar pagamento Dinheiro	Alternativo	PagamentoAdapter("DIN HEIRO").processarPagamento(30.0)	Retornar true.
CT09	Pagamento com valor zero	Exceção	processarPagamento(0)	Lançar IllegalArgumentException.
CT10	Pagamento com tipo inválido	Exceção	PagamentoAdapter("BO LETO")	Lançar IllegalArgumentException.
CT11	Criar reserva com Builder	Válido	Builder com estado definido	Criar reserva com o estado informado.
CT12	Criar reserva sem estado	Exceção	new ReservaBuilder().build()	Lançar IllegalStateException.
CT13	Alterar estado da reserva	Válido	setEstado(new ReservaConfirmada())	Estado atual deve ser ReservaConfirmada.
CT14	Notificar observer com stub	Válido	Adicionar ObserverStub e chamar notificar(...)	O stub deve receber a mensagem enviada.
CT15	Notificar dois observers com mock	Válido/extra	Dois mocks de NotificacaoObserver	Os dois mocks recebem a mensagem exatamente uma vez.

4. Arquivos de teste criados

- EstacionamentoCreatorTest.java: contém 5 testes do padrão Factory Method.
- PagamentoAdapterTest.java: contém 5 testes do padrão Adapter.
- ReservaBuilderStateObserverTest.java: contém 5 testes envolvendo Builder, State e Observer.

5. Uso de stubs e mocks

Stub: foi criado um ObserverStub para simular um observador real e guardar a última mensagem recebida. Também foi criado um EstadoResStub para testar o Builder sem depender de um estado concreto específico.

Mock: foi usado Mockito para criar objetos simulados de NotificacaoObserver e verificar se o método atualizar foi chamado exatamente uma vez para cada observador.

6. Como executar os testes

No terminal, dentro da pasta do projeto, executar:

```
mvn test
```

7. Estratégia de versionamento sugerida

Para atender ao pedido do professor, o grupo deve usar um único repositório GitHub, com participação ativa de todos os integrantes. A sugestão é usar uma branch principal main, uma branch de integração develop e uma branch individual para cada parte do trabalho.

Integrante / parte	Branch sugerida
João - diagrama	diagrama-joao
Henrique - documento arquitetural	documento-henrique
Duda - código Java e JavaDoc	codigo-duda
Pedro - testes	testes-pedro

8. Padrão de commits sugerido

- docs: adiciona documento arquitetural dos padrões GoF
- docs: atualiza diagrama de classes evoluído
- feat: implementa padrões de projeto no protótipo Java
- test: adiciona testes unitários do Factory Method
- test: adiciona testes unitários do Observer
- docs: adiciona casos de teste estruturados
- chore: configura JUnit e Mockito no pom

9. Fluxo de trabalho no Git

1. Clonar o repositório do grupo.
2. Entrar na branch develop e atualizar com git pull.
3. Criar uma branch própria para a sua parte.
4. Fazer alterações pequenas e testar.
5. Fazer commits incrementais com mensagens padronizadas.
6. Enviar a branch para o GitHub.
7. Abrir Pull Request para develop.
8. Fazer merge após a revisão do grupo.

Observação: os commits não devem ficar concentrados em uma única pessoa. Cada integrante precisa aparecer no histórico do repositório usando a própria conta do GitHub.